Projekt 1 - kryteria oceny

Poniższe kryteria służą do szczegółowej oceny rozwiązania projektu 1 (*MapReduce*, *Apache Hive*, *Apache Airflow*).

Stosuje się je zarówno przy samoocenie rozwiązania, jak i przy ocenie przykładowego projektu (wybrane z kryteriów) - zawsze w odniesieniu do zestawu danych i celów przetwarzania ocenianego rozwiązania.

Ponadto warto korzystać z tych kryteriów już podczas weryfikacji jakości zaimplementowanego przez siebie projektu - traktowane jako lista kontrolna pozwalają sprawdzić, czy rozwiązanie spełnia wszystkie wymagania, i w efekcie uzyskać rozwiązanie wysokiej jakości.

Część	**Punkty**	**Kryteria**
I. Program *MapReduce*	20	12
II. Program *Apache Hive*	20	15
III. Przepływ *Apache Airflow*	10	8
Razem	**50**	**35**

Zasady oceny

- Każde kryterium jest **binarne** - jest spełnione w całości albo wcale.
- **TAK** - kryterium spełnione, przyznaje się pełną liczbę punktów kryterium.
- **NIE** - kryterium niespełnione, 0 pkt. Podane przykłady niespełnienia nie są listą zamkniętą.
- **N/D** - kryterium nie dotyczy ocenianego zestawu (wymagana operacja w nim nie występuje), punkty przyznaje się automatycznie. Możliwość oceny N/D jest zaznaczona przy kryterium.
- Liczba punktów podana przy kryterium jest jego wagą. Maksymalna suma wynosi 50 pkt.

- Każda część ma jeden **warunek konieczny** (niepunktowany). Jeśli nie jest spełniony, cała część otrzymuje 0 pkt, niezależnie od pozostałych kryteriów.
- Kryteria oznaczone jako *wydajność* dotyczą efektywnego wykorzystania zasobów, a nie samej poprawności wyniku.

I. Program *MapReduce* (20 pkt)

WARUNEK KONIECZNY - ZGODNOŚĆ WARIANTU I REALNA IMPLEMENTACJA (NIEPUNKTOWANY)

Zanim ocenisz poniższe kryteria, zweryfikuj zgodność wariantu implementacji. Jeśli warunek nie jest spełniony, całą część *MapReduce* oceniasz na 0 pkt.

- Zadeklarowany wariant (*Classic* lub *Hadoop Streaming*) jest zgodny z faktycznie dostarczonymi plikami:
  - *Classic*: obecny jest plik JAR, a skrypt używa `hadoop jar ...`,
  - *Streaming*: obecne są pliki `mapper.py` i `reducer.py`, a skrypt używa `mapred streaming ...`.
- Implementacja zawiera realne rozwiązanie dla ocenianego zestawu – a nie przykładowy lub przypadkowy zestaw plików, który ma niewiele wspólnego z poprawnym rozwiązaniem tego zestawu.
- Implementacja nie zawiera fundamentalnych i oczywistych błędów, które uniemożliwiają jej użycie (także po dostosowaniu) albo nawet jej rzetelną ocenę.

I.A. *Mapper* - poprawność

M01 – *Mapper* prawidłowo parsuje dane wejściowe (2 pkt)

TAK, gdy:

- Rekordy są dzielone właściwym separatorem (np. przecinkiem w pliku CSV).
- Wymagane pola są odczytywane z właściwych pozycji, a tam, gdzie to konieczne, następuje konwersja typów (liczby, daty w formacie zestawu).
- Wiersz nagłówkowy pliku CSV (jeśli istnieje) jest pomijany.

NIE, gdy (przykłady):

- Błędny separator, złe indeksy pól lub brak konwersji typów powodują błędy obliczeń.

- Nagłówek trafia do obliczeń jako zwykły rekord.

M02 - Klucz pośredni (grupowania) jest kompletny i zgodny z wymaganiami zestawu (2 pkt)

TAK, gdy:

- Klucz zawiera wszystkie pola wymagane do grupowania (np. `hotel_id i year`).
- W wariancie *Streaming* separator jest spójny z konfiguracją zadania, a zakres klucza jest właściwie zdefiniowany.

NIE, gdy (przykłady):

- Brakuje wymaganego pola grupowania.
- Separator powoduje błędne grupowanie (klucz obejmuje część wartości).

M03 - *Mapper* emituje wszystkie wartości niezbędne do obliczeń i umożliwia agregację wielopoziomową (2 pkt)

TAK, gdy:

- Emitowana wartość zawiera dane potrzebne do obliczenia każdej z wymaganych miar zdefiniowanych w opisie zestawu.
- Uwzględnione są także wartości potrzebne wyłącznie do dalszych obliczeń w *Hive*.
- Dla miar niedystrybutywnych (np. średniej) *mapper* emituje **wszystkie składowe** (np. sumę i liczność osobno), a nie pojedynczą gotową wartość - dzięki temu możliwa jest późniejsza agregacja wielopoziomowa (*combiner*).

NIE, gdy (przykłady):

- Brakuje pola niezbędnego do obliczeń (np. emitowana jest tylko cena, co uniemożliwia poprawne obliczenie średniej).
- *Mapper* emituje gotową średnią - *combiner* nie mógłby działać poprawnie.

M04 – *Mapper* poprawnie obsługuje wymaganą operację złożoną (2 pkt, możliwe N/D)

TAK, gdy (zależnie od operacji wymaganej w zestawie):

- *Złożona logika daty* (np. sezon, kwartał): progi dat są poprawne, a wynik jest zgodny z definicją z opisu zestawu.
- *Wielokrotna emisja kluczy z jednego rekordu źródłowego*: *mapper* emituje osobną parę dla każdego wymaganego klucza.
- *Rozbicie pola wielowartościowego* (np. lista bohaterów rozdzielona znakiem `;`): *mapper* poprawnie dzieli pole i emituje osobny rekord dla każdego elementu.
- *Agregacja warunkowa* (np. zliczane są tylko rekordy o określonym statusie): warunek jest zaimplementowany, a rekordy niespełniające go są pomijane.

NIE, gdy (przykłady):

- Błąd w logice daty, np. błędne progi sezonu.
- Brak emisji dla jednego z wymaganych kluczy.
- Brak podziału pola wielowartościowego.
- Błędny lub brakujący warunek filtrowania.

> N/D
>
> Zestaw nie wymaga w *mapperze* żadnej operacji złożonej -> punkty przyznawane automatycznie.

I.B. *Mapper* - wydajność i odporność

M05 - *Mapper* przetwarza rekordy strumieniowo i obsługuje błędne dane (2 pkt, wydajność)

TAK, gdy:

- *Mapper* przetwarza każdy rekord i od razu emituje wynik - **bez słowników, list ani innych struktur gromadzących dane z wielu rekordów** przed emisją.
- *Mapper* nie emituje danych błędnych ani nadmiarowych (niepochodzących z bieżącego rekordu źródłowego).
- Dane wejściowe są jawnie weryfikowane (np. sprawdzenie liczby pól, obsługa wyjątku konwersji) - błędne rekordy są pomijane bez przerywania działania *mappera*.

NIE, gdy (przykłady):

- *Mapper* wczytuje rekordy do słownika lub listy przed emisją - naruszenie wzorca przetwarzania strumieniowego.

- Brak jakiejkolwiek weryfikacji - błędny rekord powoduje wyjątek i awarię zadania.
- *Mapper* emituje dane niezwiązane z bieżącym rekordem.

I.C. *Combiner* (agregator łączący)

M06 - *Combiner* jest zaimplementowany (1 pkt)

TAK, gdy:

- *Combiner* może być osobnym plikiem lub klasą albo tym samym plikiem lub klasą co *reducer* - o ile jego wynik ma format wyjścia *mappera* (patrz kolejne kryterium).
- *Streaming*: *combiner* jest wskazany w skrypcie uruchamiającym (opcja `-combiner`).
- *Classic*: *combiner* jest skonfigurowany w zadaniu.

NIE, gdy (przykłady):

- Brak *combinera* lub brak jego konfiguracji w zadaniu - cała agregacja odbywa się wyłącznie w *reducerze*.

M07 - *Combiner* poprawnie agreguje dane i zachowuje format wyjścia *mappera* (2 pkt)

TAK, gdy:

- *Combiner* poprawnie dokonuje częściowej agregacji - dla funkcji dystrybutywnych (np. `COUNT`, `SUM`, `MAX`, `MIN`) tak samo jak *reducer*, a dla niedystrybutywnych (np. `AVG`) łączy składowe (sumę i liczność), zamiast wyliczać gotowe wartości.
- *Combiner* korzysta wyłącznie z operacji przemiennych (komutatywnych) i łącznych - wynik nie zależy od kolejności ani sposobu pogrupowania łączonych wartości.
- **Format wyjściowy *combinera* jest identyczny z formatem wyjściowym *mappera*** (te same klucze, te same separatory, te same pola wartości) - tak, aby *combiner* mógł być stosowany wielokrotnie.
- *Combiner* przetwarza dane strumieniowo - bez gromadzenia w pamięci danych wielu kluczy.

NIE, gdy (przykłady):

- *Combiner* agreguje niepoprawnie, np. wylicza cząstkowe średnie zamiast łączyć sumy i liczności.
- Format wyjściowy *combinera* różni się od wyjścia *mappera* - *reducer* nie mógłby jednakowo przetwarzać wyników *combinera* i *mappera*, a *combiner* nie mógłby działać na własnych wynikach.
- *Combiner* gromadzi w pamięci dane z wielu kluczy.

I.D. *Reducer*

M08 - *Reducer* poprawnie odczytuje format danych z poprzedniego etapu (1 pkt)

TAK, gdy:

- Separator, kolejność i typy pól w *reducerze* są spójne z wyjściem *mappera* i *combinera*.
- W wariancie *Classic* deklaracje typów w *mapperze*, *reducerze* i konfiguracji zadania są wzajemnie zgodne.

NIE, gdy (przykłady):

- Inny separator niż w *mapperze* lub błędna kolejność pól.

M09 - *Reducer* poprawnie oblicza wszystkie wymagane miary (2 pkt)

TAK, gdy:

- Każda miara (liczność, suma, średnia, liczność warunkowa) jest obliczona według poprawnego wzoru.
- Dla miar niedystrybutywnych *reducer* najpierw sumuje składowe (np. sumy i liczności) ze wszystkich wartości klucza, a dopiero potem wylicza wynik (np. suma / liczność).

NIE, gdy (przykłady):

- Błędny wzór, pominięta miara lub błąd przy łączeniu danych z *combinera*.

M10 – Format wyniku *reducera* zgodny ze specyfikacją, przetwarzanie strumieniowe (2 pkt, wydajność)

TAK, gdy:

- Wynik zawiera komplet wartości zgodny ze specyfikacją zestawu.
- W wyniku nie ma zbędnych elementów uniemożliwiających jego parsowanie (nagłówków, komunikatów diagnostycznych w strumieniu wyjściowym).
- *Reducer* emituje wynik dla bieżącego klucza i przechodzi do następnego - **bez gromadzenia w pamięci rekordów wielu kluczy**.

NIE, gdy (przykłady):

- Brakuje wymaganych atrybutów lub ich format uniemożliwia załadowanie wyniku do tabeli *Hive*.
- *Reducer* gromadzi wszystkie rekordy wszystkich kluczy w jednej strukturze.

I.E. Skrypt uruchamiający i wynik końcowy

M11 - Skrypt `run_mr.sh` poprawnie konfiguruje zadanie i obsługuje parametry ścieżek (1 pkt)

TAK, gdy:

- Skrypt przyjmuje i przekazuje parametry `input_dir1` i `output_dir3`.
- *Streaming*: *mapper*, *reducer*, *combiner* (jeśli istnieje) oraz inne wymagane pliki są wskazane i dostarczone (`-files`).
- *Classic*: klasy *mappera*, *reducera*, *combinera* i konfiguracja zadania (*job*) są spójne z implementacją.

NIE, gdy (przykłady):

- Ścieżki wpisane na stałe, brak pliku w `-files` lub błędna konfiguracja klas.

M12 - Skrypt jest powtarzalny i nie wymagał poprawek przed uruchomieniem (1 pkt)

TAK, gdy:

- Przed uruchomieniem zadania skrypt usuwa katalog wynikowy w HDFS (i bez błędu obsługuje sytuację, gdy katalog nie istnieje).

- Drugie uruchomienie bez ręcznych ingerencji kończy się sukcesem.
- Skrypt nie wymagał poprawek (wynikających z błędów autora) przed pierwszym poprawnym uruchomieniem.

NIE, gdy (przykłady):

- Brak usuwania katalogu wynikowego - drugie uruchomienie kończy się błędem.
- Konieczna była ręczna edycja skryptu w celu naprawy błędów autora.

II. Program *Apache Hive* (20 pkt)

WARUNEK KONIECZNY - ZGODNOŚĆ ZESTAWU I REALNA IMPLEMENTACJA (NIEPUNKTOWANY)

Zanim ocenisz poniższe kryteria, zweryfikuj zgodność zestawu. Jeśli warunek nie jest spełniony, całą część *Hive* oceniasz na 0 pkt.

- Dostarczone pliki odpowiadają ocenianemu zestawowi (zestawowi, dla którego przygotowano rozwiązanie).
- Implementacja zawiera realne rozwiązanie dla ocenianego zestawu - a nie przykładowy lub przypadkowy zestaw plików, który ma niewiele wspólnego z poprawnym rozwiązaniem tego zestawu.
- Implementacja nie zawiera fundamentalnych i oczywistych błędów, które uniemożliwiają jej użycie (także po dostosowaniu) albo nawet jej rzetelną ocenę.

II.A. Definicja struktur danych

H01 – Tabele źródłowe poprawnie zdefiniowane - nazwy atrybutów i typy zgodne ze specyfikacją (1 pkt)

TAK, gdy:

- Tabela dla danych z *MapReduce* ma atrybuty o **nazwach zgodnych ze specyfikacją zestawu** i właściwych typach.
- Tabela dla `datasource4` odzwierciedla strukturę słownika z opisu zestawu.
- Obie tabele wskazują dane przez parametry ścieżek, a nie przez ścieżki wpisane na stałe (ewentualnie dane są do nich ładowane z miejsc wskazanych przez parametry).

NIE, gdy (przykłady):

- Nazwy atrybutów tabeli dla danych z *MapReduce* są niezgodne ze specyfikacją.
- Błędne typy uniemożliwiają poprawne obliczenia.
- Ścieżki wpisane na stałe.

H02 – Dane źródłowe dostępne przez `EXTERNAL TABLE ... LOCATION`, a nie przez `LOAD DATA` (1 pkt, wydajność)

TAK, gdy:

- Dla obu źródeł danych używane jest `CREATE EXTERNAL TABLE ... STORED AS ... LOCATION '...'`.
- Dane źródłowe nie są fizycznie kopiowane ani przenoszone instrukcjami `LOAD DATA`.

NIE, gdy (przykłady):

- Użyto `LOAD DATA INPATH` lub `LOAD DATA LOCAL INPATH` – dane są fizycznie przenoszone lub kopiowane, co jest kosztowne i narusza zasadę wskazywania danych tam, gdzie się znajdują.

H03 – Skrypt HQL jest powtarzalny – każda trwała struktura poprzedzona `DROP ... IF EXISTS` (1 pkt)

TAK, gdy:

- Przed każdą instrukcją `CREATE TABLE` / `CREATE VIEW` znajduje się `DROP ... IF EXISTS`.
- Ponowne uruchomienie skryptu nie powoduje błędów związanych z istniejącymi strukturami.

NIE, gdy (przykłady):

- Brak `DROP ... IF EXISTS` – drugie uruchomienie kończy się błędem, np. "Table already exists".

H04 – Po wykonaniu skryptu struktury danych źródłowych pozostają dostępne (1 pkt)

TAK, gdy:

- Tabele definiujące dane źródłowe (wynik *MapReduce* i `datasource4`) nie są usuwane na końcu skryptu.
- Po zakończeniu przepływu można odpytać te tabele w celu weryfikacji danych (debugowania).

NIE, gdy (przykłady):

- Skrypt na koniec usuwa tabele źródłowe - po jego uruchomieniu weryfikacja danych źródłowych jest niemożliwa.

II.B. Wydajność przetwarzania

H05 - Brak zbędnych tabel pośrednich i wielokrotnego odczytu tych samych danych (2 pkt, wydajność)

TAK, gdy:

- Dane źródłowe są odczytywane jednokrotnie - wynik uzyskiwany jest w jednym złożonym zapytaniu, z użyciem CTE lub podzapytań, w jednym przebiegu.
- Brak tabel pośrednich tworzonych wyłącznie po to, by przechować częściowy wynik dla kolejnego `SELECT`.
- Brak instrukcji `INSERT INTO tabela_posrednia SELECT ... FROM zrodlo` tworzących serię kroków przetwarzania, gdy możliwe było jedno złożone zapytanie.

NIE, gdy (przykłady):

- Skrypt tworzy kilka tabel pośrednich wypełnianych przez serię oddzielnych zadań (np. `CREATE TABLE t1 AS SELECT ... FROM source; CREATE TABLE t2 AS SELECT ... FROM t1`).
- Te same dane źródłowe są odczytywane wielokrotnie - w osobnych zapytaniach lub w obrębie jednego zapytania.
- Dane są kopiowane do nowych tabel przed przetworzeniem.

II.C. Przetwarzanie - podstawowe

H06 - Złączenie (`JOIN`) danych z wyniku *MapReduce* i `datasource4` jest poprawne (1 pkt)

TAK, gdy:

- Złączenie odbywa się na właściwym polu klucza obcego (np. `hotel_id`).
- Typ złączenia jest odpowiedni - dane bez odpowiednika w słowniku są obsługiwane zgodnie z wymaganiami zestawu.

NIE, gdy (przykłady):

- Złączenie na błędnym polu (np. na nazwie zamiast identyfikatora).
- Brak złączenia - dane ze słownika nie są dołączone.

H07 - Agregacje wykonane na wymaganym poziomie grupowania z użyciem właściwych funkcji (2 pkt)

TAK, gdy:

- `GROUP BY` zawiera wszystkie i tylko te pola, których wymaga specyfikacja zestawu.
- Funkcje agregujące są właściwe (`SUM` , `COUNT` , `AVG`), a współczynniki (np. `cancelled_ratio`) są obliczone zgodnie z definicją z opisu zestawu.

NIE, gdy (przykłady):

- Brakuje pola w `GROUP BY` lub `GROUP BY` obejmuje zbędne pola.
- Błędna funkcja agregująca lub błędne obliczenia wyników.

H08 – Wynik zawiera wszystkie wymagane atrybuty i poprawne wartości (2 pkt)

TAK, gdy:

- Wszystkie atrybuty wynikowe wymagane w opisie zestawu są obecne i mają poprawne nazwy.
- Wartości miar są poprawne i spójne (logicznie sensowne, wskaźniki w przedziale od 0 do 1, wynik ma właściwą liczbę wierszy).

NIE, gdy (przykłady):

- Brakuje atrybutów lub ich nazwy są niezgodne ze specyfikacją.

- Wartości są błędne lub logicznie niemożliwe.

II.D. Operacje zaawansowane - grupa alfa: funkcje analityczne

Grupa alfa

Każdy zestaw danych wymaga co najmniej jednej z poniższych operacji. Jeśli dana operacja nie występuje w ocenianym zestawie, kryterium oznacza się jako N/D i przyznaje punkty automatycznie.

H09 - Ranking `RANK() OVER (PARTITION BY ... ORDER BY ...)` zastosowany poprawnie (2 pkt, możliwe N/D)

TAK, gdy:

- `RANK()` (lub `ROW_NUMBER()`, jeśli jest to uzasadnione) jest użyta z właściwym `PARTITION BY` (wymagany poziom grupowania rankingu) i właściwym `ORDER BY` (właściwa miara i kierunek sortowania).
- Wynikowa kolumna rankingu jest zgodna z opisem zestawu.

NIE, gdy (przykłady):

- Błędne `PARTITION BY` lub `ORDER BY`.
- Ranking zastąpiony ręczną numeracją bez funkcji okienkowej.

> **N/D**
>
> Zestaw nie wymaga rankingu -> punkty przyznawane automatycznie.

H10 - Inna złożona funkcja okienkowa (bez funkcji rankingu), np. średnia dla grupy, odchylenie od niej lub flaga powyżej/poniżej średniej (2 pkt, możliwe N/D)

TAK, gdy:

- Funkcja okienkowa jest użyta z właściwymi argumentami, właściwym `PARTITION BY` (wymagany poziom grupowania) i - jeśli dotyczy - właściwym `ORDER BY`.

- Przykładowo: `AVG() OVER (PARTITION BY ...)` oblicza średnią na wymaganym poziomie grupowania (np. średnią dla wszystkich rekordów tej samej kategorii).
- Przykładowo: odchylenie jest różnicą między wartością bieżącej grupy a tą średnią.
- Przykładowo: flaga (`true` / `false`) poprawnie porównuje wartość z obliczoną średnią.

NIE, gdy (przykłady):

- Błędne `PARTITION BY` (np. średnia globalna zamiast średniej na wymaganym poziomie podziału).
- Odchylenie lub flaga obliczone w osobnym zapytaniu, bez funkcji okienkowej.

> **N/D**
>
> Zestaw nie wymaga innej złożonej operacji analitycznej -> punkty przyznawane automatycznie.

II.E. Operacje zaawansowane - grupa beta: dane złożone i filtry

Grupa beta

Poniższe kryteria dotyczą konkretnych operacji zdefiniowanych w opisie zestawu. Jeśli dana operacja nie jest w ocenianym zestawie wymagana, kryterium oznacza się jako N/D i przyznaje punkty automatycznie.

H11 - Rozwinięcie pola wielowartościowego (`LATERAL VIEW EXPLODE`) zastosowane poprawnie (1 pkt, możliwe N/D)

TAK, gdy:

- Pole z listą wartości rozdzielonych separatorem (np. lista gatunków, cech, produktów) jest rozwinięte przez `LATERAL VIEW EXPLODE(SPLIT(kolumna, separator))` .
- Dalsze przetwarzanie odbywa się na poziomie pojedynczych elementów listy.

NIE, gdy (przykłady):

- Pole wielowartościowe jest traktowane jako jeden ciąg znaków, brak `EXPLODE` lub błędny separator.

> N/D
>
> Zestaw nie zawiera pola wielowartościowego -> punkty przyznawane automatycznie.

H12 - Zagnieżdżona kolekcja rekordów (`ARRAY<STRUCT<...>>`) w kolumnie wynikowej (1 pkt, możliwe N/D)

TAK, gdy:

- Wymagana kolumna wynikowa (np. ranking marek) jest zbudowana jako zagnieżdżona kolekcja przez `COLLECT_LIST(NAMED_STRUCT(...))` lub równoważny mechanizm.
- Kolekcja zawiera wszystkie atrybuty zagnieżdżonych rekordów wymagane w opisie zestawu.

NIE, gdy (przykłady):

- Zagnieżdżona struktura zastąpiona prostą konkatenacją tekstu.
- Brakuje wymaganych atrybutów w zagnieżdżonych rekordach.

> N/D
>
> Zestaw nie wymaga kolekcji zagnieżdżonej -> punkty przyznawane automatycznie.

H13 - Filtrowanie wyniku na podstawie wartości analitycznej lub obliczenie udziału procentowego (1 pkt, możliwe N/D)

TAK, gdy (zależnie od operacji wymaganej w zestawie):

- *Filtr na podstawie wyniku funkcji analitycznej*: wynik jest ograniczony do wierszy, w których wartość bieżącego rekordu spełnia warunek względem wyniku funkcji analitycznej.
- *Udział procentowy*: udział jest obliczony funkcją okienkową (np. `wartosc / SUM(wartosc) OVER (PARTITION BY ...)`) na poziomie wymaganym przez

specyfikację, w skali zgodnej ze specyfikacją (ułamek od 0 do 1 lub procent).

NIE, gdy (przykłady):

- Filtr lub udział obliczony w oddzielnym zapytaniu złączonym przez `JOIN`, zamiast funkcją analityczną.
- Błędny warunek filtrowania (np. odwrócony znak porównania).

> N/D
>
> Zestaw nie wymaga żadnej z tych operacji -> punkty przyznawane automatycznie.

II.F. Skrypt uruchamiający i wynik końcowy

H14 - Skrypt `run_hive.sh` poprawnie uruchamia przetwarzanie w *Hive* i obsługuje parametry ścieżek (1 pkt)

TAK, gdy:

- Skrypt przyjmuje i przekazuje parametry `input_dir3`, `input_dir4` i `output_dir6`.
- Skrypt korzysta z `beeline`, a nie z bezpośredniego (przestarzałego) wywołania `hive`.

NIE, gdy (przykłady):

- Ścieżki wpisane na stałe.
- Brak poprawnej obsługi wymaganych parametrów.
- Użycie `hive` zamiast `beeline`.

H15 - Każdy rekord wynikowy zapisany w formacie JSON przez mechanizm SerDe (1 pkt)

TAK, gdy:

- Dane wynikowe są zapisane w katalogu HDFS wskazanym przez parametr `output_dir6`, w formacie JSON na poziomie rekordu.
- Format JSON jest generowany przez SerDe (np. `org.apache.hive.hcatalog.data.JsonSerDe` lub inny dostępny w danym

środowisku), a nie przez ręczne sklejanie ciągów znaków.

NIE, gdy (przykłady):

- Wynik w formacie CSV lub innym niż JSON.
- JSON budowany przez `CONCAT`, interpolację ciągów lub w inny "ręczny" sposób zamiast przez SerDe.

III. Przepływ *Apache Airflow* (10 pkt)

WARUNEK KONIECZNY - REALNA IMPLEMENTACJA (NIEPUNKTOWANY)

Zanim ocenisz poniższe kryteria, zweryfikuj, czy oceniany plik przepływu zawiera realną implementację. Jeśli warunek nie jest spełniony, całą część *Apache Airflow* oceniasz na 0 pkt.

- Implementacja zawiera realne rozwiązanie – nie jest to np. źródłowy plik przepływu bez zmian wynikających z wykorzystania go do orkiestracji zadań projektu.
- Implementacja nie zawiera fundamentalnych i oczywistych błędów, które uniemożliwiają jej użycie (także po dostosowaniu) albo nawet jej rzetelną ocenę.

III.A. Konfiguracja operatorów i parametrów

W01 – Parametr wariantu *MapReduce* ma wartość domyślną zgodną z implementacją (1 pkt)

TAK, gdy:

- Parametr `classic_or_streaming` ma wartość domyślną (`classic` lub `streaming`) zgodną z zaimplementowanym wariantem.

NIE, gdy (przykłady):

- Wartość domyślna jest niezgodna - przepływ domyślnie uruchamia niewłaściwy wariant.

W02 – Polecenie operatora *MapReduce* zgodne ze skryptem i poprawnie używające parametrów ścieżek (1 pkt)

TAK, gdy:

- Polecenie w operatorze (`mapreduce_classic` lub `hadoop_streaming`) jest funkcjonalnie zgodne ze skryptem `run_mr.sh` .
- Ścieżki są przekazywane przez parametry: `{{ params.dags_home }}` , `{{ params.input_dir }}` , `{{ params.output_mr_dir }}` .

NIE, gdy (przykłady):

- Polecenie jest niezgodne ze skryptem lub ścieżki są wpisane na stałe.
- Operator wywołuje skrypt `run_mr.sh` zamiast zawierać polecenie uruchamiające zadanie.

W03 – Polecenie operatora *Hive* zgodne ze skryptem i poprawnie używające wszystkich parametrów ścieżek (1 pkt)

TAK, gdy:

- Polecenie w operatorze `hive` jest zgodne ze skryptem `run_hive.sh` .
- Wszystkie trzy wymagane ścieżki (`output_mr_dir` , `input_dir` , `output_dir`) są przekazywane przez parametry przepływu.

NIE, gdy (przykłady):

- Brakuje ścieżki, ścieżka jest wpisana na stałe lub polecenie jest niezgodne ze skryptem.

III.B. Integralność struktury przepływu (DAG)

W04 – Elementy niewymagające modyfikacji pozostają niezmienione lub ich zmiany są uzasadnione (1 pkt)

TAK, gdy:

- Operatory `clean_output_mr_dir` , `clean_output_dir` , `pick_classic_or_streaming` i `get_output` są niezmienione.
- Zależności między operatorami (kolejność wykonania) są niezmienione.
- Ewentualne zmiany w tych elementach są uzasadnione i nie zaburzają logiki przepływu.

NIE, gdy (przykłady):

- Zmieniono operatory czyszczące lub operator wyboru wariantu bez uzasadnienia albo zmodyfikowano zależności.

III.C. Wydajność wykonania

W05 - Przepływ nie zawiera zbędnych operatorów ani operacji niepotrzebnie zużywających zasoby (1 pkt, wydajność)

TAK, gdy:

- Brak dodatkowych operatorów pobierających dane wejściowe, rozpakowujących archiwa, ładujących dane itp. (dane źródłowe są z założenia dostępne we wskazanym miejscu).
- Brak zbędnych operacji blokujących lub powielających pracę wykonywaną już przez istniejące operatory.

NIE, gdy (przykłady):

- Dodano operatory pobierające dane źródłowe lub wykonujące operacje niewymagane przez specyfikację przepływu.
- Przepływ wykonuje zbędne operacje, np. kopiowanie danych do HDFS, które powinno zostać wykonane przed jego uruchomieniem.

III.D. Poprawność przetwarzania

W06 - Przepływ wykonuje się w całości bez błędów - wszystkie taski kończą się sukcesem (2 pkt)

TAK, gdy:

- Po uruchomieniu z interfejsu *Airflow UI* wszystkie taski uzyskują status *Success* - bez błędów po stronie autora.
- W trakcie wykonania nie jest potrzebna ręczna interwencja.

NIE, gdy (przykłady):

- Jeden lub więcej tasków kończy się statusem *Failed* lub wymaga ręcznej naprawy (z powodu błędów po stronie autora).

W07 - Przepływ jest powtarzalny - drugie uruchomienie kończy się sukcesem bez ingerencji (1 pkt)

TAK, gdy:

- Ponowne uruchomienie całego przepływu bez ręcznych operacji kończy się sukcesem.
- Operatory czyszczące HDFS działają poprawnie przy ponownym uruchomieniu.

NIE, gdy (przykłady):

- Drugie uruchomienie wymaga ręcznego usunięcia katalogów lub tabel przed startem.

W08 - Task `get_output` prezentuje w logu poprawny wynik końcowy przetwarzania (2 pkt)

TAK, gdy:

- W logu tasku `get_output` widoczne są rekordy JSON będące wynikiem przetwarzania.
- Liczba i zawartość rekordów są logicznie sensowne (wynik nie jest pusty i nie zawiera wyłącznie komunikatów o błędach).

NIE, gdy (przykłady):

- Log jest pusty lub zawiera tylko błędy.
- Dane w logu nie są w formacie JSON.

Podsumowanie

Maksymalnie **50 pkt**:

- 35 kryteriów binarnych
  - *MapReduce* - 12 kryteriów - 20 pkt,
  - *Hive* - 15 kryteriów - 20 pkt,
  - *Apache Airflow* - 8 kryteriów - 10 pkt
- oraz 3 niepunktowane warunki konieczne - po jednym dla każdej części.

Uwaga dotycząca punktacji

Skala 0–50 pkt służy wyłącznie do ustrukturyzowania oceny – pokazuje, jaką wagę mają poszczególne elementy rozwiązania. Nie jest to liczba punktów, którą bezpośrednio otrzymuje się za samoocenę ani za ocenę przykładowego projektu.

Docelowa punktacja tych aktywności wynika z zasad ustalonych na początku kursu i opisanych na jego stronie. Przykładowo za samoocenę można otrzymać punkty dodatnie, ale także ujemne, jeśli samoocena okaże się nieadekwatna do rzeczywistej zawartości projektu.